RELEASE IMPACT · HOW IT REASONS

Does a release change a route the old (BAU) app still uses?

A new release ships new routes for the new app — but the production app keeps calling its own older route. Release Impact answers one question: did this release's commits also change *that older route*? Here is exactly how it decides, using the same route-resolution the framework uses at runtime.

1 The one rule — how a version resolves to a route

When an app calls an API, the framework picks the route by version: the **exact** versioned route if it exists, otherwise the **immediate lower** version (the highest route below it). Same rule, two apps:

New app
client `19.14` calls `/sg/manage/addPayee` →

uses `R9.14` **if it exists**, else falls back to `R9.10`

BAU app
client `19.10` (in production) calls `/sg/manage/addPayee` →

uses `R9.10` — **always**, even when R9.14 exists

> Key point for the release team: the new app may move to `R9.14`, but the live app **never stops calling** `R9.10`. So `R9.10` is the "BAU route" — and any change to it is a production change.

2 Two cases — which route does each app land on?

Case A · `R9.14` was added

The release adds a new versioned route.

- New app → `R9.14` (new work lives here)
- BAU app → `R9.10` (unchanged, still live)

New fields on `R9.14` are **new-app-only** — not a BAU risk. **But** we still inspect `R9.10`: if a commit in this release also touched it, that **is** a production change (blast radius).

Case B · no `R9.14`

The API has no route at the new version.

- New app → falls back to `R9.10`
- BAU app → `R9.10`

Both apps share `R9.10`. Any change this release makes to `R9.10` hits **everyone** — highest-attention scope.

> In both cases the question is identical: **did this release change `R9.10`?** That is what we compute next — and it's checked even when the new app doesn't use it.

3 How we conclude “BAU changed / not changed”

For the BAU route `R9.10` we look at two things it owns — its Camel **route XML** and the **request template** (`.ftl` / `.vm`) it sends — and diff each against **its own previous version**, counting only this release’s commits.

STEP 1 · PICK THE RELEASE’S COMMITS

Take the commits whose message carries the exact release token `[19.14.0]` — matched exactly, so `19.4.0` and `19.10.0` never count.

↓

STEP 2 · DIFF R9.10 AGAINST ITS PREVIOUS FILE VERSION

For each such commit, `git diff` the route’s XML *and* its template against the file’s **previous version in that file’s history** — **whitespace, blank lines and CR/LF ignored** (same as IntelliJ’s “ignore whitespaces”). A reformat or line-ending change shows nothing.

↓

STEP 3 · IS EITHER DIFF NON-EMPTY?

Not changed Both diffs empty → the release didn’t touch R9.10. **No BAU impact.**

BAU changed A structural step **or** a payload line changed → **High risk + backward-compat**, attributed to the commit author.

> Why “previous *file* version”, not the commit’s parent? Because with merges or unrelated commits in between, the graph parent can look different. Diffing against the file’s own prior revision is exactly what you see in the IntelliJ file history — so the result matches what a reviewer verifies by hand.

4 Worked example

✓ Counts — a 19.14.0 change to R9.10

R9.10 sends `addpayeeConfirm.ftl`. A `[19.14.0]` commit adds a field.

```
META-INF/templates/sg/manage/addpayeeConfirm.ftl
  "channelId": "${channel}",
+ "tokenSerialNumber": "${token}",
  "bankAddress": {…}
```

BAU changed · High · BC by the commit author

✗ Doesn't count — a different version's change

The same file's `bankAddress.addressLine2` was changed — but by a `[19.4.0]` commit.

```
…changed by commit tagged [19.4.0], not [19.14.0]
- "addressLine2": "value1"
+ "addressLine2": "value2"
```

Excluded

not a 19.14.0 commit → not this release's impact

> Only the release's own commits, diffed against each file's previous version, decide the verdict — so a field an *earlier* release changed never gets attributed to this one.

5 What the Release Impact tab shows

- **BAU-route changes lead** — they appear first, under a ⚑ **BAU route modified** group, ahead of new-version work.
- Each card carries the change kind (**Payload change** / Route change / Route removed), a **BC** marker, and the **commit author** ("who to ask").
- The **actual git diff** is shown (whitespace-ignored), collapsed by default with a **± context** toggle — the exact lines the release changed, nothing else.
- **Summary** view = one line per API for leads; **Detailed** view = the full diffs. Both export to PDF.

Route resolution = exact version, else immediate lower · BAU route = the older route the production app still calls · "BAU changed" = a 19.14.0 commit altered that route's XML or its request template, diffed against the file's own previous version, whitespace/line-endings ignored.